

PaceUp

Guía de inicio de la app móvil en local

App Flutter · backend running-api ya desplegado

1. Qué es y contra qué corre

PaceUp es la app móvil de running en Flutter: plan de entrenamiento, seguimiento de carreras en vivo con GPS, historial persistente e inscripción a maratones.

El backend ya está levantado. No hace falta montar nada de servidor para trabajar en la app:

URL base	https://cam-run.tumype.com/api/v1
Estado	Operativo. GET /health y GET /api/v1/config/app responden 200
Moneda / zona	BOB · America/La_Paz
Autenticación	JWT. El login acepta correo o CI en el mismo campo

Auth, Home y el catálogo de maratones hablan ya con el backend real. Train, Races y Profile todavía usan repositorios en memoria (ver ARCHITECTURE.md, sección 9).

Si el catálogo sale vacío o el login da 401, la base de producción está sin sembrar. Se arregla en el VPS con un solo comando: `cd /srv/running-api && npm run db:seed`. Mientras tanto puedes crear una cuenta desde la pantalla de registro de la app.

2. Requisitos

Flutter	3.44 o superior (Dart SDK ^3.12.2)
Android	Android SDK con compileSdk 37. Emulador o teléfono físico
iOS	Xcode (solo si vas a compilar para iPhone)
Web	Chrome instalado
Red	Salida a Internet hacia cam-run.tumype.com

Comprueba el entorno antes de nada:

```
flutter --version
flutter doctor
```

Dependencias principales que ya vienen en pubspec.yaml y conviene conocer: flutter_riverpod (estado y DI), go_router (navegación), dio (red), hive_ce y drift (persistencia local), geolocator y flutter_map (GPS y mapas), flutter_secure_storage (tokens), fl_chart (gráficos).

3. Arranque en cuatro pasos

```
git clone <repo> && cd running-app
flutter pub get
flutter devices          # que aparezca tu emulador, telefono o Chrome
make run                 # movil, contra el backend de produccion
```

Al arrancar por primera vez verás el onboarding. Entra con `runner@test.com / Test1234!` — es la única cuenta sembrada con datos (entrenamientos, plan activo, inscripciones y zapatillas). El resto están en `docs/cuentas-de-prueba.md`.

4. El puerto 5000 en web (obligatorio)

Para correr la app en Chrome hay un solo comando válido:

```
make run-web
# equivale a:
# flutter run -d chrome --web-port=5000 --dart-define-from-file=.env
```

El puerto va fijo en 5000 a propósito, no es un capricho. El navegador exige que el origen desde el que se hacen las peticiones esté autorizado por el servidor mediante CORS, y el backend solo tiene en su lista blanca `http://localhost:5000`. Si dejas que `flutter run -d chrome` elija puerto, elegirá uno distinto cada vez, y un puerto aleatorio no se puede meter en una lista blanca.

Del lado del servidor, en `/etc/running-api/.env.production`:

```
CORS_ORIGINS=https://cam-run.tumype.com,http://localhost:5000
```

```
sudo systemctl restart running-api
```

Síntoma cuando falta. La app se queda en el esqueleto de carga: el servidor responde 200 y el navegador tira la respuesta. Comprobación rápida —debe devolver la cabecera:

```
curl -sI -H "Origin: http://localhost:5000" \
https://cam-run.tumype.com/api/v1/config/app | grep -i access-control-allow-origin
```

Si el puerto 5000 está ocupado en tu máquina, libéralo; cambiarlo en el comando rompe las peticiones hasta que alguien añada el nuevo origen en el VPS.

5. A qué backend apunta cada comando

La URL base **no se escribe en el código**: entra como constante de compilación desde un archivo de la raíz del repo, con `--dart-define-from-file`, que es de Flutter y no necesita ningún paquete.

Comando	Archivo	URL base
<code>make run</code>	<code>.env</code>	<code>https://cam-run.tumype.com/api/v1</code>
<code>make run-web</code>	<code>.env</code>	<code>https://cam-run.tumype.com/api/v1</code> (puerto 5000)
<code>make run-local</code>	<code>.env.local</code>	<code>http://10.0.2.2:3000/api/v1</code>

Lo que hay en esos archivos se **incrusta en el binario**: ahí no va nunca un secreto, cualquiera con el APK puede leerlo.

Desde un teléfono físico contra un backend local, `10.0.2.2` no sirve—solo existe dentro del emulador de Android—; usa la IP del equipo en la red:

```
flutter run --dart-define=API_BASE_URL=http://192.168.1.50:3000/api/v1
```

Sin ningún archivo, la app cae al backend local por defecto (`api_config.dart`).

Levantar el backend en tu máquina (opcional)

Solo si necesitas tocar la API. Lo normal es usar el servidor ya desplegado.

```
cd ../running-api
npm install && npx prisma migrate deploy && npm run db:seed
npm run dev                # escucha en :3000
```

y en otra terminal, `make run-local`. Sin `db:seed` no hay maratones ni cuentas de prueba: la app arranca, pero no hay con qué entrar.

6. Comandos disponibles

```
make run           # movil contra produccion
make run-web       # Chrome contra produccion, puerto 5000
make run-local     # movil contra localhost:3000
make apk           # APK de release (con --dart-define-from-file=.env)
make fmt           # dart format .
make analyze       # flutter analyze
make test          # flutter test
make goldens       # flutter test --update-goldens
```

Qué comando según el sistema operativo

Los targets del Makefile son atajos de una línea. En **Linux** y **macOS** `make` viene de serie y son los comandos normales; en **Windows**, PowerShell no trae `make`, así que se llama a flutter directamente.

Sistema	Arrancar en Chrome (puerto 5000)
Linux	<code>make run-web</code>
macOS	<code>make run-web</code>
Windows	<code>flutter run -d chrome --web-port=5000 --dart-define-from-file=.env</code>

En macOS make llega con las Command Line Tools de Xcode; si faltan: `xcode-select --install`. En Linux, `sudo apt install make` (o el equivalente de tu distro).

Equivalencia completa target → comando directo, válida en cualquier sistema:

Target (Linux / macOS)	Comando directo (Windows y cualquier sistema)
make run	<code>flutter run --dart-define-from-file=.env</code>
make run-web	<code>flutter run -d chrome --web-port=5000 --dart-define-from-file=.env</code>
make run-local	<code>flutter run --dart-define-from-file=.env.local</code>
make apk	<code>flutter build apk --release --dart-define-from-file=.env</code>
make fmt	<code>dart format .</code>
make analyze	<code>flutter analyze</code>
make test	<code>flutter test</code>
make goldens	<code>flutter test --update-goldens</code>

Solo en macOS, además, se puede compilar para iPhone:

```
flutter run -d ios --dart-define-from-file=.env
flutter build ipa --release --dart-define-from-file=.env
```

En Windows, si prefieres usar el Makefile tal cual, MSYS2 trae mingw32-make (C:\msys64\mingw64\bin): añade esa carpeta al PATH y llama `mingw32-make run-web`.

7. Qué esperar de la app

Home	Cuenta atrás del próximo maratón, tarjeta del evento, plan semanal con anillos de progreso y sesión del día
Train	Inicio rápido, resumen semanal, historial agrupado y filtrable
Races	Totales calculados, inscripciones próximas y completadas con resultados
Profile	Estadísticas, calzado, sueño, hidratación y ajustes

Flujos completos disponibles: onboarding y auth (sign in / sign up / recuperar contraseña); detalle de maratón e inscripción en 3 pasos con dorsal; sesión de running a pantalla completa con mapa, splits y auto-pausa; resumen post-entrenamiento persistido; perfil editable. Tema claro y oscuro conmutables desde **Profile** → **Appearance**.

Probar una carrera sin salir a correr. En modo debug LocationService usa SimulatedLocationService, que reproduce una ruta pregrabada 20 veces más rápido. Para GPS real en un dispositivo, sobrescribe useSimulatedLocationProvider con false.

Catálogo de componentes. En debug, navega a `/dev/showcase`.

8. Limitaciones conocidas en web

Grabación en segundo plano	No funciona: el GPS del navegador se para al cambiar de pestaña
Tokens	Van a localStorage, no al Keychain: vale para mirar pantallas, no para juzgar seguridad

Mapas y resto de UI	Igual que en el teléfono
----------------------------	--------------------------

Web sirve para ver pantallas rápido. Lo que se prueba de verdad —tracking, permisos, segundo plano— se prueba en un teléfono.

9. Problemas frecuentes

Pantalla de carga infinita en Chrome	CORS. Confirma que arrancaste con <code>make run-web</code> (puerto 5000) y que el origen está en <code>CORS_ORIGINS</code>
401 INVALID_CREDENTIALS	La base no está sembrada, o la contraseña no es <code>Test1234!</code>
Catálogo de maratones vacío	Falta <code>npm run db:seed</code> en el backend
El teléfono físico no conecta	Estás usando <code>10.0.2.2</code> , que solo existe en el emulador. Pasa la IP del equipo por <code>--dart-define</code>
Errores raros tras cambiar de rama	<code>flutter clean && flutter pub get</code>

10. Documentación relacionada

- `README.md` — resumen del proyecto y puesta en marcha.
- `docs/cuentas-de-prueba.md` — cuentas sembradas, entornos y estado de producción.
- `ARCHITECTURE.md` — capas, convenciones y cómo añadir una feature.